

Name: Aziz Sayyad

Roll No: 381069

Batch: A3

Assignment No: 1

Problem Statement:

Perform tokenization (Whitespace, Punctuation-based, Treebank, Tweet, MWE) using NLTK library. Use Porter Stemmer and Snowball Stemmer for stemming. Use any technique for lemmatization.

Objective:

- To understand the concept of tokenization and its various types.
- To implement Whitespace, Punctuation-based, Treebank, Tweet, and MWE tokenizers using NLTK.
- To understand and apply stemming techniques using Porter Stemmer and Snowball Stemmer.
- To understand lemmatization and apply it using NLTK's WordNet Lemmatizer.
- To compare the outputs of different tokenization, stemming, and lemmatization methods.

S/W Packages and H/W apparatus used:

- Operating System: Windows/Linux/macOS
- Kernel: Python 3.x
- Tools: Google Colab / Jupyter Notebook
- Hardware: CPU with minimum 4GB RAM
- Libraries and packages used: NLTK, Python string module

Theory:

Natural Language Processing (NLP) is a branch of AI that enables computers to understand, interpret, and generate human language. Before applying any NLP model or algorithm, raw text must be preprocessed. The three fundamental preprocessing steps are tokenization, stemming, and lemmatization.

Tokenization:

Tokenization is the process of splitting text into smaller units called tokens (words, sentences, or subwords). NLTK provides several tokenizers:

- Whitespace Tokenizer: Splits text on whitespace characters. Simple but does not handle punctuation.
- Punctuation-based Tokenizer: Splits on whitespace and punctuation, treating each punctuation mark as a separate token.
- Treebank Tokenizer (word_tokenize): Standard tokenizer based on Penn Treebank conventions. Handles contractions and punctuation intelligently.
- Tweet Tokenizer: Designed for social media text; preserves hashtags, mentions, emoticons, and handles elongated words.
- MWE (Multi-Word Expression) Tokenizer: Groups predefined multi-word expressions (e.g., 'New York') into single tokens.

Stemming:

Stemming reduces words to their root/base form by removing suffixes using heuristic rules. The result may not always be a valid word.

- Porter Stemmer: One of the oldest and most widely used stemming algorithms. Uses a series of rules to iteratively strip suffixes. Fast but can be over-aggressive.
- Snowball Stemmer (Porter2): An improved version of the Porter Stemmer. Supports multiple languages and is more accurate and consistent.

Lemmatization:

Lemmatization reduces words to their dictionary base form (lemma) using vocabulary and morphological analysis. Unlike stemming, the result is always a valid word. NLTK's WordNetLemmatizer uses the WordNet lexical database. It requires a part-of-speech (POS) tag for best results (e.g., 'better' -> 'good' when tagged as adjective).

Methodology:

- Import the NLTK library and download required resources (punkt, wordnet, averaged_perceptron_tagger).
- Define a sample input text for all experiments.
- Apply Whitespace Tokenizer by splitting text on spaces.
- Apply Punctuation-based Tokenizer using RegexpTokenizer to split on non-alphanumeric characters.
- Apply Treebank Tokenizer (word_tokenize) and observe handling of contractions and punctuation.
- Apply Tweet Tokenizer and observe preservation of hashtags, mentions, and emoticons.
- Define MWE phrases, apply MWETokenizer to group multi-word expressions into single tokens.
- Apply Porter Stemmer to a list of words and print the stemmed forms.
- Apply Snowball Stemmer (English) to the same word list and compare results with Porter.
- Apply WordNet Lemmatizer to words with appropriate POS tags and print the lemmas.
- Compare and analyze the outputs of all techniques.

Use Cases:

- Use Case 1 - Tokenization for Text Classification: Whitespace and Punctuation-based tokenizers preprocess raw text into tokens for bag-of-words models used in spam detection and document classification.
- Use Case 2 - Social Media Analysis: Tweet Tokenizer processes Twitter/X posts for sentiment analysis while preserving hashtags, mentions, and emoticons that carry contextual meaning.
- Use Case 3 - Search Engine Indexing: Porter and Snowball stemmers combined with lemmatization normalize words during indexing, allowing search engines to match queries like 'running' with documents containing 'run' or 'runs'.

Advantages:

- NLTK provides ready-to-use implementations of all major tokenizers, stemmers, and lemmatizers.
- Multiple tokenization strategies allow choosing the best fit for a specific text domain.
- Stemming is computationally fast and reduces vocabulary size effectively.

- Lemmatization produces linguistically valid base forms, improving model interpretability.
- Tweet Tokenizer handles informal and noisy social media text efficiently.

Limitations:

- Whitespace tokenization fails to handle punctuation attached to words.
- Stemming can produce non-dictionary words (over-stemming or under-stemming).
- Lemmatization requires correct POS tags; incorrect tags lead to wrong lemmas.
- MWE Tokenizer requires manually defining all multi-word expressions in advance.
- NLTK is slower compared to modern NLP libraries like spaCy for large-scale processing.

Applications:

- Search engines: Stemming and lemmatization normalize queries and documents for better matching.
- Sentiment analysis: Tokenization is the first step in opinion mining pipelines.
- Machine translation: Tokenization prepares source text for translation models.
- Social media analytics: Tweet Tokenizer enables processing of Twitter/X data.
- Chatbots and virtual assistants: Lemmatization helps understand user intent from varied inputs.

Conclusion:

This assignment explored fundamental NLP preprocessing techniques using the NLTK library. We implemented five types of tokenizers -Whitespace, Punctuation-based, Treebank, Tweet, and MWE - each suited to different types of text. We applied Porter and Snowball stemmers to reduce words to their root forms and used the WordNet Lemmatizer for linguistically accurate base-form extraction. Understanding and correctly applying these techniques is essential for building robust NLP pipelines, as the quality of preprocessing directly impacts downstream model performance.